GPU-Accelerated Graph-Colored Simulated Annealing for Integer Factorization

Advith Desu, Aryan Namboodri, and Anil Prabhakar
Centre for Quantum Information, Communication and Computing,
Indian Institute of Technology Madras, Chennai 600036, India
(Dated: May 25, 2026)

The security of RSA rests on the hardness of factoring a semiprime $N = PQ$. Shor's algorithm dissolves that hardness, but only at qubit counts that remain decades away. In the interim, classical and quantum-inspired Ising machines offer a near-term lever on the same problem. We present an end-to-end pipeline that maps factorization to a sparse Ising model, solves it on a single NVIDIA GH200 GPU with a graph-colored simulated annealer, and closes the residual gap with a guided brute-force search. The solver's three structural choices — a hub-tagged sparse CSR backed by a shared-memory hub cache, dense/sparse spin binning matched to GPU resources, and Welsh–Powell coloring of the interaction graph for exact parallel Metropolis — together factor a $\sim$ 100-bit semiprime in 416 seconds end-to-end, an 8$\times$ improvement over a $\sqrt{N}$-seeded brute-force baseline on the same hardware.

I. INTRODUCTION

Public-key cryptography assumes that some computational problems are asymptotically hard. RSA in particular assumes that recovering the two prime factors of a public modulus $N = PQ$ takes time exponential in the bit length of N. Shor's algorithm collapses this assumption on a fault-tolerant quantum computer, but the resource counts — on the order of millions of physical qubits for cryptographically relevant key sizes [1] — place that threat at least a decade out. Meanwhile, classical and quantum-inspired Ising machines have matured to the point where they routinely solve quadratic unconstrained binary optimization (QUBO) problems with 10^4–10^5 variables, and recent work has factored integers as large as 2049 bits on specialized hardware [2].

Integer factorization can be cast as a QUBO [3, 4]. The cleanest route is to write $N = PQ$ as a binary long multiplication, treat the unknown bits of P, Q and the carries as binary variables, square the per-column constraints, and minimize the resulting energy. The ground state encodes the factors. Whether that ground state can actually be *found* depends on three things: how clean the resulting QUBO is, how well the solver exploits its sparsity, and how the solver scales on the hardware it runs on.

This paper is about the second two. We treat the QUBO construction as support infrastructure and focus on a GPU-resident simulated annealer that has been shaped, end-to-end, by the structure of the Ising graphs that factorization produces. Three design choices carry the contribution:

1. **Graph-colored parallel Metropolis.** A Welsh–Powell coloring of the interaction graph partitions spins into independent sets. Within a color, every spin's ΔE can be computed and applied in parallel without invalidating the global energy accumulator.

2. **Dense/sparse spin binning.** Empirically, only a small set of "hub" spins (with row degree $\gtrsim 100$) attracts most of the coupling weight, while the rest of the graph stays sparse. The annealer routes the two populations through different kernels — block-level shared-memory reduction for hubs, warp-level shuffle reduction for sparse spins — sized to the GPU's actual resources.

3. **Hub-tagged sparse CSR.** The high bit of each column index in the sparse-spin CSR encodes whether the neighbor is a hub (read from a shared-memory cache) or a non-hub (read from global memory). The hot path is a single branchless shared-memory load.

Together, these let one GH200 sweep a $\sim 10^5$-spin Ising graph at ~ 0.1 ms per iteration, and factor a $\sim$ 100-bit semiprime end-to-end in 416 s.

Section II casts factorization as a binary optimization problem and works through a small example. Section III describes the classical simplification and quadrization that turns it into a sparse Ising model. Section IV shows how the structure of that model scales, which motivates the kernel design in Section V. Section VI describes the post-processing search that closes the residual gap. Section VII reports timing and solution-quality results across 16–128-bit semiprimes, and Section VIII discusses limits and next steps.

II. FROM FACTORING TO A BINARY ENERGY

Let $N = PQ$ be a semiprime with P, Q odd primes of equal bit length $n = \lceil \frac{1}{2} \log_2 N \rceil$. Write

$$P = \sum_{k=0}^{n-1} 2^k p_k, \qquad Q = \sum_{k=0}^{n-1} 2^k q_k, \qquad p_k, q_k \in \{0, 1\}. \tag{1}$$

Four bits are known a priori: $p_0 = q_0 = 1$ (both factors are odd) and $p_{n-1} = q_{n-1} = 1$ (each is exactly n bits).

The product PQ, expanded as a binary long multiplication, places the cross-term $p_a q_b$ in column $a + b$. Car-

ries propagate left; we introduce binary carry variables $s_{i,j} \in \{0,1\}$, where $s_{i,j}$ is the bit of column i's carry destined for column $j > i$ (with weight 2^{j-i} when it arrives). Equating the value of column i in PQ to the value of column i in N yields a per-column constraint:

$$C_i \;=\; N_i \;-\; \sum_j q_j p_{i-j} \;-\; \sum_j s_{j,i} \;+\; \sum_j 2^j s_{i,i+j} \;=\; 0, \tag{2}$$

where N_i is the ith bit of N. The correct factor assignment satisfies $C_i = 0$ for every column. Squaring and summing turns the constraint system into an unconstrained optimization:

$$E(\{p_i, q_i, s_{ij}\}) \;=\; \sum_i C_i^2. \tag{3}$$

The ground state $E = 0$ recovers the factors.

a. Worked example, $N = 391 = 17 \times 23$. With $n = 5$, the unknown factor bits are p_1, p_2, p_3 and q_1, q_2, q_3 — six free bits. The carry variables span at most two columns ahead (each column sum is bounded by a small constant), so the full system has nine column constraints $C_0, \ldots, C_8$ and a handful of carry variables. Three of those constraints are immediately useful: C_0 is trivially satisfied ($1-1=0$); C_8 reduces to $s_{6,8}+s_{7,8}=0$, forcing both carries to zero; and C_1 reads $p_1 + q_1 = 1 + 2s_{1,2}$, which has no solution with $s_{1,2} = 1$ (the LHS is at most 2) so $s_{1,2} = 0$ and $q_1 = 1 - p_1$. The same logic threads through C_2. Six rules of this kind — enumerated next — handle the bulk of the simplification.

III. CLASSICAL PRE-PROCESSING

Squaring (2) directly produces a polynomial with *many* more terms than needed: each carry variable shows up in multiple clauses, and cross-products like $p_i q_j$ appear with very different weights in different columns. Before handing the problem to the annealer, we eliminate as many variables and as much redundancy as possible using deterministic algebraic rules [3].

A. Reduction rules

Each $C_i = 0$ is a linear (or low-degree) equation over $\{0,1\}$ variables. Six rules suffice to drive most of the carries and several of the factor bits to fixed values; their formal statements and short proofs are collected in Appendix A.

Saturated sum.: $\sum x_i = n$ (all coefficients equal ± 1, n terms) forces every $x_i = 1$.

Zero sum.: $\sum x_i = 0$ with same-sign coefficients forces every $x_i = 0$.

Doubling.: $x_1 + x_2 = 2x_3$ forces $x_1 = x_2 = x_3$.

Coefficient bound.: If a single coefficient exceeds the achievable opposing sum, that term is forced to 0 or 1.

Complement.: $x_1 + x_2 = 1$ gives $x_2 = 1 - x_1$ and $x_1 x_2 = 0$, eliminating a variable and killing one quadratic term in any clause that referenced both.

Parity.: Both sides of a column clause must have the same parity; this pins the parity of the small factor-bit sum against the (always-even) carry-out terms.

The simplifier applies these rules in a fixed priority order, looping until no rule fires on any clause. A seventh *Replacement* rule isolates an s-variable that appears with coefficient ± 1 and substitutes it back into every other clause. Replacement is powerful but greedy: a long chain of cascading replacements can fuse many small clauses into a single mega-clause whose squaring blows up the QUBO. We guard against this with a *checkpoint*: the simplifier saves its state before the first Replacement and restores it if the chain failed to produce new value assignments.

B. Quadrization

The energy (3) is degree four in the binary variables. Cubic and quartic terms arise from squaring clauses that contain products like $p_i q_j$. Standard QUBO solvers (and Ising hardware) require degree two, so we apply *Rosenberg substitutions* [5]: introduce an auxiliary w and add a penalty $P(A, B, w) = M(AB - 2Aw - 2Bw + 3w)$ with M large enough that any minimizer has $w = AB$. After substitution every cubic $AB \cdot s$ becomes a quadratic $w \cdot s$ plus a constant-size quadratic penalty. The two sign variants of the cubic case, and the two variants of the quartic case (negative quartic needs only a single auxiliary; positive quartic needs two and is handled recursively), are standard. Implementation details and coefficient bookkeeping follow [3].

C. From QUBO to Ising

A linear change of variables $\sigma_i = 1 - 2x_i$ maps each binary $x_i \in \{0,1\}$ to an Ising spin $\sigma_i \in \{-1, +1\}$. The QUBO $H = \sum_i Q_{ii} x_i + \sum_{i<j} Q_{ij} x_i x_j$ becomes

$$H(\boldsymbol{\sigma}) = \sum_i h_i \sigma_i + \sum_{i<j} J_{ij} \sigma_i \sigma_j + \text{const.}, \tag{4}$$

with, treating Q symmetrically so $Q_{ij} = Q_{ji}$ for $i \neq j$,

$$J_{ij} = \tfrac{1}{4} Q_{ij}, \quad h_i = -\tfrac{1}{2} Q_{ii} - \tfrac{1}{4} \sum_{j \neq i} Q_{ij}. \tag{5}$$

The Ising form is what every downstream piece consumes.

IV. STRUCTURAL SCALING AND SPARSE STORAGE

What does the resulting Ising problem actually look like? Two empirical facts shape the rest of the paper.

a. The coupling matrix is sparse, and stays sparse. Figure 1 plots the nonzero structure of J_{ij} for four representative semiprimes. Two visual features dominate every panel: a small dense block of "hub" rows / columns near index zero, and a long thin diagonal trail. The dense block corresponds to auxiliary w-variables introduced during quadrization (Section III B), each of which couples to every clause it participates in; the diagonal trail comes from the carries $s_{i,j}$, which couple only to their immediate column neighbors.

b. Hub spin count grows only with $\log N$. Figure 2 separates the total spin count from the *dense* spin count (row degree ≥ 128). The former grows roughly cubically with n; the latter grows only logarithmically, and saturates around 120 at 128 bits. The dense set is small enough to hold in shared memory in its entirety, and large enough that every sparse spin reads from it on every sweep. This is the structural fact that the kernel design in Section V exploits.

c. CSR storage. A dense $|J|^2$ representation is wasteful at $|J| > 10^4$ and infeasible at $|J| > 10^5$. We store J in Compressed Sparse Row (CSR) format: a `row_ptr` of $N+1$ row offsets, a `col_idx` array of column indices, and a `values` array of the nonzeros. Memory is $\mathcal{O}(N + \text{nnz})$ rather than $\mathcal{O}(N^2)$; for the problems considered here, nnz is typically 10–100$\times$ smaller than N^2.

V. GRAPH-COLORED SIMULATED ANNEALING ON THE GPU

The annealer is a Metropolis sampler that tries to flip every spin once per sweep, accepting flips with probability $\min(1, e^{-\beta \Delta E_i})$. The serial cost of one sweep is $\mathcal{O}(\text{nnz})$. To turn that into wall-clock time on a GPU we need three things: a way to flip many spins in parallel without breaking the energy bookkeeping, kernels whose work pattern matches GPU resources, and a memory layout that keeps the hot data in fast storage.

A. Why parallel Metropolis needs coloring

Single-spin Metropolis updates the state one spin at a time. Two spins coupled by $J_{ij} \neq 0$ cannot be flipped simultaneously: the ΔE each one computes depends on the current value of the other, so flipping both at once leaves the running energy accumulator inconsistent with the new spin state.

Graph coloring of the interaction graph (nodes = spins, edges = nonzero J_{ij}) partitions the spins into independent sets such that no two same-color spins are coupled. All spins of one color can then be flipped in parallel: each one's ΔE depends only on spins of *other* colors, which are unchanged during this color's pass. Sweeping every color in sequence gives a full Metropolis sweep with no synchronization inside a color and one synchronization between colors.

We compute the coloring once at setup using a Welsh–Powell greedy heuristic (largest-degree-first, first-fit). The number of colors is upper-bounded by $\Delta + 1$ where Δ is the maximum degree; empirically the heuristic produces colorings within a few colors of optimum.

B. Dense and sparse spin bins

The two empirical features from Section IV — a small dense hub set and a long sparse tail — map naturally onto two GPU kernels.

Dense bin: (row degree ≥ 128). One CUDA block of 1024 threads handles one spin. Each thread accumulates a partial sum over a stride of the spin's neighbor row; a shared-memory tree reduction collapses the partials to ΔE. The block size matches the dense-row length so threads stay busy.

Sparse bin: (row degree < 128). One warp (32 threads) handles one spin; 32 spins are packed into a 1024-thread block. Each warp does a warp-shuffle reduction — no shared memory, no explicit synchronization within the warp. This gives much higher occupancy than the dense layout would for short rows.

The threshold 128 matches the warp's natural reduction tier and is robust across the bit-width range we tested: above it, dense reduction amortizes the block-launch overhead; below it, the warp layout wins on occupancy.

C. Hub-tagged sparse CSR

The dense (hub) spins are read by essentially every sparse spin in every color pass. Reading them from global memory each time is expensive; we cache them once per color pass.

Hub values for the entire problem are copied into a shared-memory array (at most 256 bytes total) at the start of each sparse kernel launch. To resolve "is this neighbor a hub or not?" branchlessly, we *tag* the column-index array of the sparse-only CSR: the high bit of `col_idx[k]` encodes the answer.

- High bit set: the low 31 bits are an index into the hub cache in shared memory.
- High bit clear: the entry is a raw global spin id, used as a fallback for the rare non-hub neighbor.

FIG: Sparsity of J_{ij} at $N = 39\,203$ (162 spins), 159 197 (379), 338 699 (563), and 377 977 (902). Most nonzeros concentrate in a small band along the top rows / left columns, with a thin diagonal tail. Pattern persists across orders of magnitude in N.

FIG. 1. *[placeholder]* Sparsity of J_{ij} at $N = 39\,203$ (162 spins), 159 197 (379), 338 699 (563), and 377 977 (902). Most nonzeros concentrate in a small band along the top rows / left columns, with a thin diagonal tail. Pattern persists across orders of magnitude in N.

FIG: Number of total Ising spins versus number of densely connected *(row degree ≥ 128) hub spins as the bit width of N grows from* 16 *to* ~ 128*. Total grows as $\mathcal{O}(n^3)$; hub count grows as $\mathcal{O}(\log N)$, plateauing near* 120 *at* 128 *bits.*

FIG. 2. *[placeholder]* Number of total Ising spins versus number of *densely connected* (row degree ≥ 128) hub spins as the bit width of N grows from 16 to ~ 128. Total grows as $\mathcal{O}(n^3)$; hub count grows as $\mathcal{O}(\log N)$, plateauing near 120 at 128 bits.

The inner neighbor-sum loop is therefore a sign test on `col_idx[k]`, a single shared-memory load on the hot path, and a global-memory load on the cold path. Whenever a hub spin flips (which can only happen on a color pass that contains hubs), the hub-value cache is refreshed by a small dedicated kernel before any sparse pass reads it.

D. Auto start temperature and cooling

A simulated-annealing run is parameterized by a temperature schedule $T_t = T_0 \alpha^t$ with $0 < \alpha < 1$ and an inner sweep count. The wrong T_0 wastes sweeps: too high and the run is effectively a random walk for many iterations; too low and the system freezes into the initial basin.

We estimate T_0 adaptively, in the style of Ben-Ameur [6]. Across $K = 10$ random spin configurations we sample every spin's uphill ΔE (the energy cost of a single flip when it raises the energy), average them, and set

$$T_0 \;=\; \langle \Delta E^{+} \rangle \,/\, \ln(1/\chi), \tag{6}$$

where $\chi = 0.5$ is the target acceptance rate at the start of the schedule. The sampling reuses the same GPU buffers and hub cache as the annealing loop, so the cost is at most one full sweep per configuration. Cooling is geometric with $\alpha = 0.95$; the schedule runs until $T < 10^{-3}$.

E. Numerical scaffolding

Two correctness checks and one bookkeeping invariant keep the run honest.

Symmetry check.: On load, the annealer samples random rows of the CSR and verifies that for every stored (i, j, J_{ij}) the partner (j, i, J_{ji}) is also stored and has the same value. The energy and ΔE formulas assume this; an upper-triangular-only CSR would silently halve every off-diagonal coupling.

Running-energy accumulator.: Instead of recomputing the total energy from scratch after each sweep, the annealer keeps a running sum: every accepted flip atomically adds its ΔE to a `double` accumulator. At the end of each annealing run, the accumulator is checked against a fresh full-state recomputation; relative drift stays below 10^{-9} for all problem sizes we tested.

Best-state mirror.: A second device-side spin buffer mirrors the live state whenever a new best running energy is observed. The output spin file is written from this mirror, so the reported state is the best one ever visited, not whatever happened to be live at the final T step.

F. Putting it together

Algorithm 1 lists the one-time setup; the per-temperature loop is Algorithm 2.

Algorithm 1: One-time setup before annealing

```
Verify J symmetry and zero diagonal (sampled);
Bin each spin: dense if d_i ≥ 128, else sparse;
color[·] ← Welsh–Powell greedy coloring
    (largest-degree-first, first-fit);
Bucket spins by color into dense / sparse offset arrays;
Build the hub-tagged sparse-only CSR;
Upload CSR, color tables, and hub arrays to the GPU;
Initialize a random spin state and compute the initial
    energy;
Estimate T_0 = ⟨ΔE^+⟩ / ln(1/χ);
```

Algorithm 2: Color-parallel simulated annealing loop

```
foreach β in geometric schedule do
    for sweep = 1 ... m do
        Generate fresh uniform randoms for all N
          spins;
        for c = 0 ... N_colors − 1 do
            Launch collectFlipCandidates⟨dense⟩
              on color c;
            Launch collectFlipCandidates⟨sparse⟩
              on color c;
            applyAllFlipsInColor: flip every
              accepted spin, atomically add ΔE_i to the
              energy accumulator;
            if color c contained any hub then
                Refresh the hub-value cache;
    if running energy < best energy then
        snapshot best spins;
return best-energy spin configuration;
```

VI. BRUTE-FORCE POST-PROCESSING

The annealer rarely lands exactly on the ground state at the bit widths we test. It does, however, land *close*: post-processing of its spin output recovers approximate factors $(P_{\text{pred}}, Q_{\text{pred}})$ that differ from the true (P, Q) by a few percent (Section VII). A targeted brute-force search closes the residual gap.

a. Centering. The annealer provides two independent estimates of P: P_{pred} from the spins directly, and $\lfloor N/Q_{\text{pred}} \rfloor$ from the complementary factor. Their average is the search centroid:

$$P_{\text{centre}} \;=\; \tfrac{1}{2}\big(P_{\text{pred}} + \lfloor N/Q_{\text{pred}} \rfloor\big). \tag{7}$$

Empirically, averaging halves the typical gap (see Figure 5).

b. Outward-spiral search. Candidates are tested in chunks of $C = 2^{14}$, in outward-spiral order: chunk k covers $[P_{\text{centre}} + \frac{k}{2}C, P_{\text{centre}} + (\frac{k}{2}+1)C - 1]$ if k is even (upward step) and the analogous downward window if k is odd. Threads claim the next chunk via an atomic counter, so the search is partially parallel and the closest candidates to P_{centre} are always tested first.

FIG: SA wall-clock per sweep iteration versus number of Ising spins, comparing the CUDA kernel on GH200, a PyTorch port on GH200, and the same PyTorch code on an Apple M2 Max via MPS. CUDA stays near 0.05 ms across four decades of spin count; the PyTorch variants drift upward past 10^3 spins.

FIG. 3. *[placeholder]* SA wall-clock per sweep iteration versus number of Ising spins, comparing the CUDA kernel on GH200, a PyTorch port on GH200, and the same PyTorch code on an Apple M2 Max via MPS. CUDA stays near 0.05 ms across four decades of spin count; the PyTorch variants drift upward past 10^3 spins.

c. Wheel sieve. Within a chunk, candidates that are divisible by 2, 3, 5, 7, or 11 are skipped without a divisibility test against N. The wheel of modulus $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 = 2310$ has 480 residues coprime to all five small primes — roughly 21% of 2310 — so the sieve filters out about 79% of candidates at essentially zero cost.

d. Early termination. The first thread to find $p \mid N$ sets a shared stop flag; all other threads exit at the next chunk boundary. The expected wall-clock cost follows the empirical model

$$T_{\text{bf}} \;\approx\; \frac{3.32 \times 10^{-9}}{\text{threads}} \;\cdot\; \text{Gap}(\%) \;\cdot\; 2^{n}, \tag{8}$$

where n is the bit length of the (smaller) factor and Gap(%) is the relative distance between P_{centre} and the true P. The exponential in n is unavoidable; the only knob the annealer controls is the multiplicative prefactor.

VII. RESULTS

All experiments run on a single NVIDIA GH200 Grace Hopper Superchip (Hopper architecture, sm_90). The annealing kernel is compiled with `nvcc` (CUDA 12.x); QUBO construction and brute-force post-processing run on the Grace CPU side. Cooling schedule: geometric with $\alpha = 0.95$, T_0 auto-estimated at $\chi = 0.5$, $T_{\min} = 10^{-3}$, 10 sweeps per β step.

For each bit width $n \in \{16, 20, 24, \ldots, 124, 128\}$ we generate a random n-bit semiprime $N = PQ$ with P, Q random $\lceil n/2 \rceil$-bit primes, build the QUBO once, and run the SA ten times with independent seeds. Reported numbers use the winning (lowest-energy) seed; mean and spread are reported where useful.

A. Per-iteration throughput

Figure 3 compares wall-clock per sweep across the three implementations. The CUDA kernel stays near

FIG: Time to build J and time to run one full annealing schedule, both as a function of n in $[16, 128]$ bits. Construction is sub-second up to ~ 60 bits and reaches $\sim 60\,s$ at 128 bits. Annealing dominates above ~ 60 bits, reaching $\sim 220\,s$ at 128 bits.

FIG. 4. *[placeholder]* Time to build J and time to run one full annealing schedule, both as a function of n in $[16, 128]$ bits. Construction is sub-second up to ~ 60 bits and reaches $\sim 60\,\mathrm{s}$ at 128 bits. Annealing dominates above ~ 60 bits, reaching $\sim 220\,\mathrm{s}$ at 128 bits.

FIG: Percentage gap between predicted and true P as a function of n, for three estimators: $\sqrt{N}$ (uniform baseline), P_{pred} from SA, and the averaged centroid $\frac{1}{2}(P_{\mathrm{pred}} + N/Q_{\mathrm{pred}})$. Means across the tested range: 10.33% for $\sqrt{N}$, 4.23% for raw SA, and 2.98% for the centroid.

FIG. 5. *[placeholder]* Percentage gap between predicted and true P as a function of n, for three estimators: $\sqrt{N}$ (uniform baseline), P_{pred} from SA, and the averaged centroid $\frac{1}{2}(P_{\mathrm{pred}} + N/Q_{\mathrm{pred}})$. Means across the tested range: 10.33% for $\sqrt{N}$, 4.23% for raw SA, and 2.98% for the centroid.

0.05 ms per iteration across four orders of magnitude in spin count; the PyTorch port pays roughly 5× that on the same GPU because its gather/scatter primitives cannot exploit the hub cache. The Mac MPS curve drifts upward earlier and faster, consistent with its lower memory bandwidth.

B. End-to-end construction + anneal time

Figure 4 plots both wall-clock components against bit width. QUBO construction is dominated by the algebraic simplification at small n and by the squaring + quadrization at large n; SA wall-clock scales roughly linearly with the spin count, which itself grows as $\mathcal{O}(n^3)$.

C. Solution quality

Figure 5 reports the relative gap between the predicted and the true factor for three estimators:

1. $\sqrt{N}$ — the uniform "I know nothing about the factors" baseline. Mean gap 10.33% over the tested range.
2. P_{pred} from the SA spin state directly. Mean gap 4.23%.

FIG: Time taken to find the exact factor by the wheel-sieve brute-force search, as a function of the raw gap between P_{centre} and the true P, on a 72-thread CPU. Below a $\sim 10^5$ gap the search completes in milliseconds (sieve dominated); above $\sim 10^7$ it follows the $\mathrm{Gap} \cdot 2^n$ prediction of Eq. (8).

FIG. 6. *[placeholder]* Time taken to find the exact factor by the wheel-sieve brute-force search, as a function of the raw gap between P_{centre} and the true P, on a 72-thread CPU. Below a $\sim 10^5$ gap the search completes in milliseconds (sieve dominated); above $\sim 10^7$ it follows the $\mathrm{Gap} \cdot 2^n$ prediction of Eq. (8).

3. The averaged centroid $\frac{1}{2}(P_{\mathrm{pred}} + \lfloor N/Q_{\mathrm{pred}} \rfloor)$. Mean gap 2.98%.

Two facts matter for the next step. The SA output is consistently better than $\sqrt{N}$, but only by a small constant factor; and the free averaging step gains as much again. Both contribute the same multiplicative factor to the brute-force prefactor in Eq. (8).

D. Brute-force time as a function of gap

Figure 6 validates the timing model of Eq. (8). Below a gap of $\sim 10^5$ the search is sieve-dominated and finishes in milliseconds; above $\sim 10^7$ it scales linearly with the raw gap as predicted.

E. ~ 100-bit end-to-end case study

For

$$N = 416\,749\,328\,744\,899\,275\,664\,046\,210\,629,$$

the SA-guided pipeline reports:

Stage	Wall-clock
Build J matrix (CSR)	25.3 s
Simulated annealing ($\alpha = 0.95$, auto T_0)	122.8 s
Brute-force from $P_{\mathrm{centre}} = 5.79 \times 10^{14}$	267.9 s
Total	**416.0 s**

The naive $\sqrt{N}$-seeded baseline on the same 72-thread CPU takes 3323.8 s, i.e. the SA pre-step buys an 8× speedup end-to-end. The recovered factors are $P = 573\,858\,364\,692\,161$ and $Q = 726\,223\,323\,360\,389$.

VIII. DISCUSSION

A. Where the time goes

Across the bit range we tested, three regimes dominate end-to-end wall-clock at different scales:

- Below ~ 40 bits, the simplifier resolves the problem completely; no SA is required.
- Between ~ 40 and ~ 80 bits, SA dominates and the brute-force step is sub-second.
- Above ~ 80 bits, the multiplicative $\text{Gap} \cdot 2^n$ factor of Eq. (8) drives the brute-force step past the SA step.

The third regime is the operational bottleneck: SA hardware is hardware-bounded, but the brute-force prefactor is purely a function of solution quality. Cutting the gap in half halves the time at every bit width.

B. Limitations

Several caveats are worth being explicit about.

a. Solution-quality gap grows with n. The mean gap of 4.23% (raw SA) is across 16–128 bits; the individual-instance gap is noisier at the upper end and we observed runs with $> 8\%$ gap at 120+ bits. The $\sim$ 100-bit case study is representative, not best-case.

b. 128-bit hard cap. The QUBO construction code uses `__uint128_t` arithmetic throughout. Going beyond 128 bits requires switching to a multi-word big-integer representation, with predictable but nontrivial code changes.

c. Trial division is the wrong post-processing tool. At cryptographic key sizes (1024+ bits), a $\text{Gap} \cdot 2^n$ brute-force cost is hopeless even with perfect predictions. The right question — which we leave to future work — is what factor-finding algorithm best exploits a P_{centre} within a few percent of the truth. Pollard's ρ and Lenstra's elliptic curve method are both candidates that benefit much more from a good starting point than linear scan does.

d. Single-instance scope. Each pipeline run targets one N. There is no notion of solving a batch of independent semiprimes in one schedule, even though the GH200 has the memory for it. Batched annealing is a straightforward extension we have not yet attempted.

e. This is not an RSA break. The largest semiprime factored here is ~ 100 bits; production RSA moduli are 2048 bits or larger. The point of the work is to measure how a carefully designed classical annealer scales on commodity GPU hardware, not to threaten deployed cryptography.

C. Future work

The bottleneck identified in Section VIII B suggests several directions. *Better QUBO encodings*: the simplification in Section III is purely algebraic; symmetry-aware or sieve-aware reductions might collapse far more of the problem before SA sees it. *Smarter post-processing*: Pollard's ρ, Lenstra ECM, or even GNFS sieving seeded by P_{centre} would exploit the annealer's output far more effectively than linear scan does. *Hardware co-design*: the kernel structure described in Section V maps naturally onto coherent Ising machines and FPGA-based annealers, both of which provide higher coupling-update throughput than a GPU for the same energy budget.

IX. CONCLUSION

A GPU-resident simulated annealer, structured around the empirical sparsity of factorization-derived Ising graphs, factors $\sim$ 100-bit semiprimes in minutes on a single GH200. The contribution is not a break of any cryptographic system, but a measurement of how far commodity classical hardware reaches against a problem usually framed in quantum terms, and a concrete kernel design — graph-colored Metropolis with hub-tagged sparse CSR — that other Ising-problem domains can pick up directly.

Appendix A: Reduction-rule proofs

For $x_i \in \{0, 1\}$:

a. Saturated sum. If $\sum_{i=1}^{n} x_i = n$ then every $x_i = 1$, since n binary variables sum to n only when all are 1.

b. Zero sum. If $\sum_{i=1}^{n} x_i = 0$ with same-sign coefficients then every $x_i = 0$, since a sum of non-negative binaries vanishes only when every term does.

c. Doubling. $x_1 + x_2 = 2x_3$ forces $x_1 = x_2 = x_3$: the LHS is in $\{0, 1, 2\}$ and the RHS in $\{0, 2\}$, so the LHS must be even; the only way two binaries sum to an even number is both equal, and that shared value matches x_3.

d. Coefficient bound. In a clause $\sum_i c_i x_i + C = 0$ with $S_+ = \sum_{c_i > 0} c_i$ and $S_- = \sum_{c_j < 0} |c_j|$: a positive coefficient c_i with $c_i > S_- - C$ forces $x_i = 0$ (otherwise the LHS cannot reach zero), and $c_i > S_+ + C$ forces $x_i = 1$. The negative-coefficient cases are symmetric.

e. Complement. $x_1 + x_2 = 1$ has only two solutions $(0, 1)$ and $(1, 0)$, so $x_2 = 1 - x_1$ and $x_1 x_2 = 0$.

f. Parity. Both sides of any linear clause must agree mod 2. In a column clause, all carry terms have even coefficients, so the parity of the small factor-bit sum is pinned by the parity of N_i. Two factor bits with odd coefficients must therefore either be equal (if N_i is even) or complementary (if N_i is odd).

g. Power rule (used by squaring). For any $x \in \{0, 1\}$ and $m \geq 1$, $x^m = x$. This makes the squared energy

expressible as a degree-2 QUBO once cubic and quartic terms are quadrized.

[1] N. Jain *et al.*, Information-theoretic and computationally secure cryptography in the quantum era, Contemporary Physics **57**, 366 (2016).

[2] X. Guo *et al.*, Scaling integer factorization on photonic Ising machines to 2049 bits, IEEE Transactions on Computers **75**, 1446 (2026).

[3] S. Jiang, K. A. Britt, A. J. McCaskey, T. S. Humble, and S. Kais, Quantum annealing for prime factorization, Scientific Reports **8**, 10.1038/s41598-018-36058-z (2018).

[4] B. Wang, F. Hu, H. Yao, and C. Wang, Prime factorization algorithm based on parameter optimization of Ising model, Scientific Reports **10**, 10.1038/s41598-020-62802-5 (2020).

[5] I. G. Rosenberg, Reduction of bivalent maximization to the quadratic case, Cahiers du Centre d'Études de Recherche Opérationnelle **17**, 71 (1975).

[6] W. Ben-Ameur, Computing the initial temperature of simulated annealing, Computational Optimization and Applications **29**, 369 (2004).